

UNIVERSIDAD NACIONAL DE COLOMBIA

SEDE BOGOTÁ

ACTIVIDAD 3

Programación Orientada a Objetos

Estudiante: Emily Ruiz Becerra
PROFESOR: Walter Hugo Arboleda Mazo
https://github.com/toffeexy/Programaci-n-Orientada-a-objetos-2026/tree/main

Ejercicio 8.2

```
import math
import tkinter as tk
from tkinter import messagebox

COLOR_FONDO = "#e6acf1"

class Notas:
    CANTIDAD_NOTAS = 5
    def __init__(self):
        self.lista_notas = [0.0] * Notas.CANTIDAD_NOTAS
    def calcular_promedio(self):
        suma = 0.0
        for nota in self.lista_notas:
            suma += nota
        return suma / len(self.lista_notas)
    def calcular_desviacion(self):
        promedio = self.calcular_promedio()
        suma = 0.0
        for nota in self.lista_notas:
            suma += math.pow(nota - promedio, 2)
        return math.sqrt(suma / len(self.lista_notas))
    def calcular_menor(self):
        menor = self.lista_notas[0]
        for nota in self.lista_notas:
            if nota < menor:
                menor = nota
        return menor
    def calcular_mayor(self):
        mayor = self.lista_notas[0]
        for nota in self.lista_notas:
            if nota > mayor:
                mayor = nota
        return mayor

class VentanaPrincipal(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Notas")
        self.geometry("280x380")
        self.resizable(False, False)
        self.configure(bg=COLOR_FONDO)
        self._centrar_ventana(280, 380)
        self.inicio()
    def _centrar_ventana(self, ancho, alto):
        x = (self.winfo_screenwidth() - ancho) // 2
        y = (self.winfo_screenheight() - alto) // 2
        self.geometry(f"{ancho}x{alto}+{x}+{y}")
```

```
    def inicio(self):
        self.campos_notas = []
        for i in range(Notas.CANTIDAD_NOTAS):
            y = 20 + i * 30
            etiqueta = tk.Label(self, text=f"Nota {i + 1}:", anchor="w", bg=COLOR_FONDO)
            etiqueta.place(x=20, y=y, width=80, height=23)
            campo = tk.Entry(self)
            campo.place(x=105, y=y, width=135, height=23)
            self.campos_notas.append(campo)

        self.boton_calcular = tk.Button(self, text="Calcular", command=self.calcular)
        self.boton_calcular.place(x=20, y=170, width=100, height=23)
        self.boton_limpiar = tk.Button(self, text="Limpiar", command=self.limpiar)
        self.boton_limpiar.place(x=125, y=170, width=80, height=23)
        self.promedio = tk.Label(self, text="Promedio = ", anchor="w", bg=COLOR_FONDO)
        self.promedio.place(x=20, y=210, width=235, height=23)
        self.desviacion = tk.Label(self, text="Desviación estándar = ", anchor="w",
bg=COLOR_FONDO)
        self.desviacion.place(x=20, y=240, width=235, height=23)
        self.mayor = tk.Label(self, text="Nota mayor = ", anchor="w", bg=COLOR_FONDO)
        self.mayor.place(x=20, y=270, width=235, height=23)
        self.menor = tk.Label(self, text="Nota menor = ", anchor="w", bg=COLOR_FONDO)
        self.menor.place(x=20, y=300, width=235, height=23)

    def calcular(self):
        notas = Notas()
        for i, campo in enumerate(self.campos_notas):
            texto = campo.get().strip()
            if texto == "":
                messagebox.showwarning("Dato faltante", f"La Nota {i + 1} no ha sido
ingresada.\n""Debe ingresar las cinco notas.")
                campo.focus_set()
                return
            try:
                valor = float(texto.replace(",", "."))
            except ValueError:
                messagebox.showerror("Dato no válido", f"La Nota {i + 1} ('{texto}') no es un valor
numérico.\n""Ingrese solo números.")
                campo.focus_set()
                campo.select_range(0, tk.END)
                return
            if valor != valor or valor in (float("inf"), float("-inf")):
                messagebox.showerror("Dato no válido", f"La Nota {i + 1} ('{texto}') no es un valor
numérico válido.")
                campo.focus_set()
                return

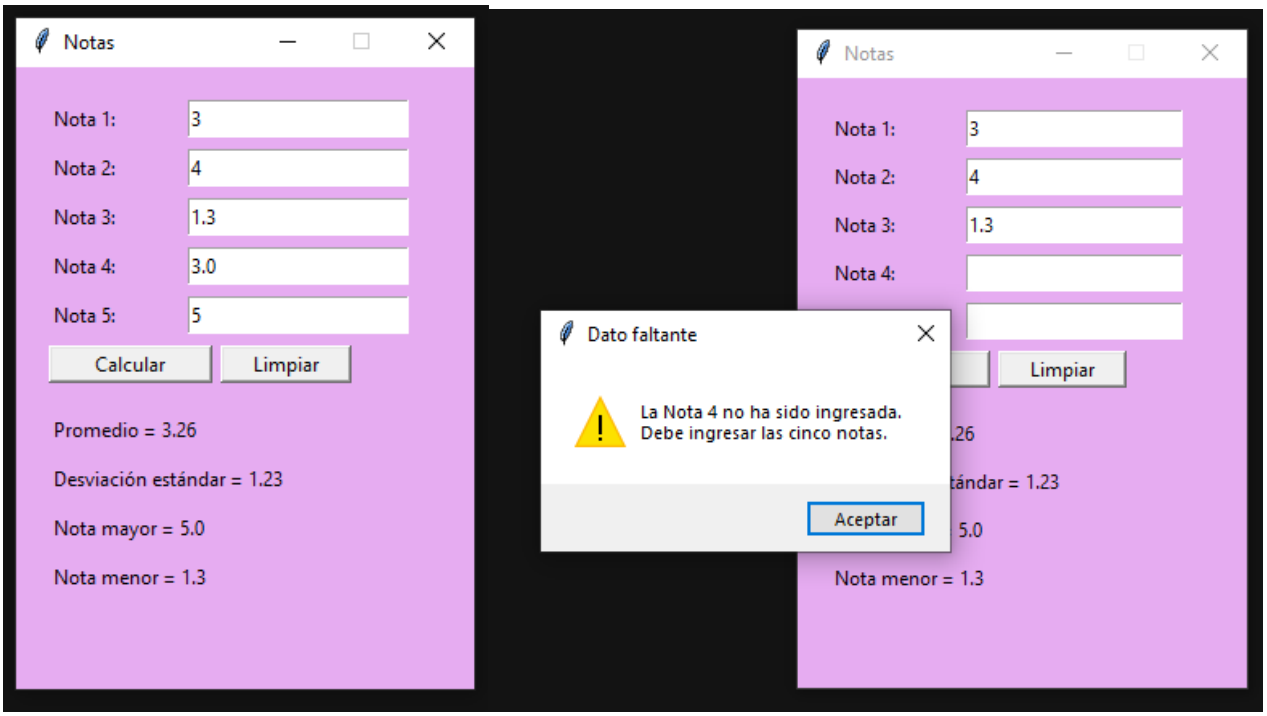
            notas.lista_notas[i] = valor
        self.promedio.config(text=f"Promedio = {notas.calcular_promedio():.2f}")
        self.desviacion.config(text=f"Desviación estándar = {notas.calcular_desviacion():.2f}")
        self.mayor.config(text=f"Nota mayor = {notas.calcular_mayor()}")
```

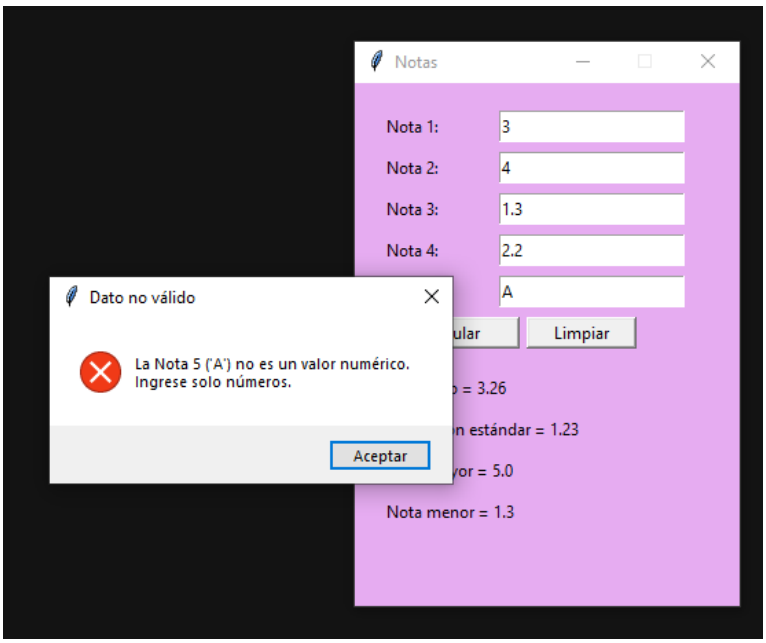
```
        self.menor.config(text=f"Nota menor = {notas.calcular_menor()}")

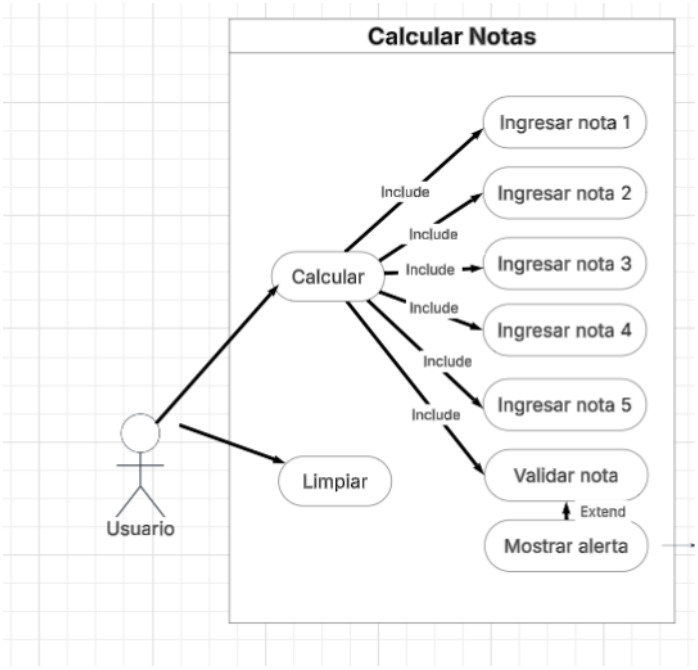
    def limpiar(self):
        for campo in self.campos_notas:
            campo.delete(0, tk.END)
        self.promedio.config(text="Promedio = ")
        self.desviacion.config(text="Desviación estándar = ")
        self.mayor.config(text="Nota mayor = ")
        self.menor.config(text="Nota menor = ")
        self.campos_notas[0].focus_set()

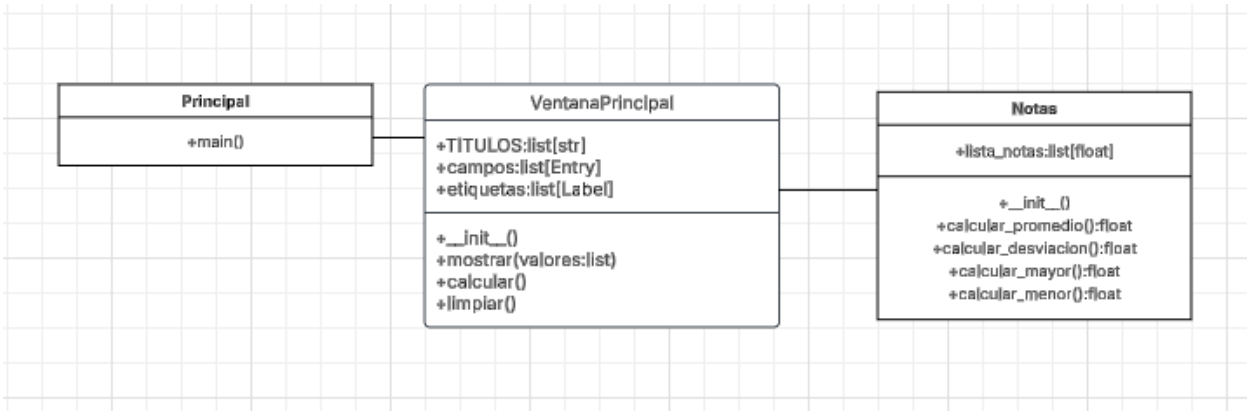
class Principal:
    @staticmethod
    def main():
        mi_ventana_principal = VentanaPrincipal()
        mi_ventana_principal.mainloop()

if __name__ == "__main__":
    Principal.main()
```




Notas
Nota 1:
3
Nota 2:
4
Nota 3:
1.3
Nota 4:
2.2
A
Limpiar
= 3.26
n estándar = 1.23
yor = 5.0
Nota menor = 1.3
Dato no válido
La Nota 5 ('A') no es un valor numérico.
Ingrese solo números.
Aceptar


Calcular Notas
Ingresar nota 1
Ingresar nota 2
Ingresar nota 3
Ingresar nota 4
Ingresar nota 5
Validar nota
Mostrar alerta
Calcular
Limpiar
Usuario
Include
Extend


Principal
+main()
VentanaPrincipal
+TITULOS:list[str]
+campos:list[Entry]
+etiquetas:list[Label]
+__init__()
+mostrar(valores:list)
+calcular()
+limpiar()
Notas
+lista_notas:list[float]
+__init__()
+calcular_promedio():float
+calcular_desviacion():float
+calcular_mayor():float
+calcular_menor():float



Ejercicio 8.3

```
import math
import tkinter as tk
from tkinter import messagebox

COLOR_FONDO = "#e6acf1"
RELLENO = "#fa8de2"
CLARO = "#d576fa"
OSCURO = "#b44fe2"
BORDE = "#6b1e8a"

class FiguraGeometrica:
    NOMBRE = ""
    DATOS = []
    def __init__(self):
        self.volumen = 0.0
        self.superficie = 0.0
    def calcular(self):
        self.volumen = self.calcular_volumen()
        self.superficie = self.calcular_superficie()
    def calcular_volumen(self):
        raise NotImplementedError
    def calcular_superficie(self):
        raise NotImplementedError
    @staticmethod
    def dibujar(lienzo):
        raise NotImplementedError

class Cilindro(FiguraGeometrica):
    NOMBRE = "Cilindro"
    DATOS = ["Radio (cms)", "Altura (cms)"]
    def __init__(self, radio, altura):
        super().__init__()
        self.radio = radio
        self.altura = altura
```

```
        self.calcular()
    def calcular_volumen(self):
        return math.pi * self.altura * math.pow(self.radio, 2)
    def calcular_superficie(self):
        return (2 * math.pi * self.radio * self.altura + 2 * math.pi * math.pow(self.radio, 2))
    @staticmethod
    def dibujar(lienzo):
        lienzo.create_oval(100, 65, 180, 95, fill=RELLENO, outline=BORDE)
        lienzo.create_rectangle(101, 25, 179, 80, fill=RELLENO, outline=RELLENO)
        lienzo.create_line(100, 25, 100, 80, fill=BORDE)
        lienzo.create_line(180, 25, 180, 80, fill=BORDE)
        lienzo.create_oval(100, 15, 180, 35, fill=CLARO, outline=BORDE)

class Esfera(FiguraGeometrica):
    NOMBRE = "Esfera"
    DATOS = ["Radio (cms)"]
    def __init__(self, radio):
        super().__init__()
        self.radio = radio
        self.calcular()
    def calcular_volumen(self):
        return 4 / 3 * math.pi * math.pow(self.radio, 3)
    def calcular_superficie(self):
        return 4 * math.pi * math.pow(self.radio, 2)
    @staticmethod
    def dibujar(lienzo):
        lienzo.create_oval(96, 6, 184, 94, fill=RELLENO, outline=BORDE)
        lienzo.create_oval(96, 38, 184, 62, outline=BORDE)

class Piramide(FiguraGeometrica):
    NOMBRE = "Pirámide"
    DATOS = ["Base (cms)", "Altura (cms)", "Apotema (cms)"]
    def __init__(self, base, altura, apotema):
        super().__init__()
        self.base = base
        self.altura = altura
        self.apotema = apotema
        self.calcular()
    def calcular_volumen(self):
        return math.pow(self.base, 2) * self.altura / 3
    def calcular_superficie(self):
        return math.pow(self.base, 2) + 2 * self.base * self.apotema
    @staticmethod
    def dibujar(lienzo):
        lienzo.create_polygon(140, 10, 90, 85, 190, 85, fill=RELLENO, outline=BORDE)
        lienzo.create_polygon(140, 10, 190, 85, 225, 65, fill=OSCURO, outline=BORDE)

class Cubo(FiguraGeometrica):
    NOMBRE = "Cubo"
    DATOS = ["Arista (cms)"]
    def __init__(self, arista):
```

```
        super().__init__()
        self.arista = arista
        self.calcular()
    def calcular_volumen(self):
        return math.pow(self.arista, 3)
    def calcular_superficie(self):
        return 6 * math.pow(self.arista, 2)
    @staticmethod
    def dibujar(lienzo):
        lienzo.create_rectangle(100, 35, 160, 95, fill=RELLENO, outline=BORDE)
        lienzo.create_polygon(100, 35, 160, 35, 185, 15, 125, 15, fill=CLARO, outline=BORDE)
        lienzo.create_polygon(160, 35, 185, 15, 185, 75, 160, 95, fill=OSCURO, outline=BORDE)

class Prisma(FiguraGeometrica):
    NOMBRE = "Prisma"
    DATOS = ["Largo (cms)", "Ancho (cms)", "Altura (cms)"]
    def __init__(self, largo, ancho, altura):
        super().__init__()
        self.largo = largo
        self.ancho = ancho
        self.altura = altura
        self.calcular()
    def calcular_volumen(self):
        return self.largo * self.ancho * self.altura
    def calcular_superficie(self):
        return 2 * (self.largo * self.ancho + self.largo * self.altura + self.ancho * self.altura)
    @staticmethod
    def dibujar(lienzo):
        lienzo.create_rectangle(80, 35, 170, 95, fill=RELLENO, outline=BORDE)
        lienzo.create_polygon(80, 35, 170, 35, 205, 15, 115, 15, fill=CLARO, outline=BORDE)
        lienzo.create_polygon(170, 35, 205, 15, 205, 75, 170, 95, fill=OSCURO, outline=BORDE)

class VentanaFigura(tk.Toplevel):
    def __init__(self, padre, clase_figura):
        super().__init__(padre)
        self.clase_figura = clase_figura
        n = len(clase_figura.DATOS)
        self.title(clase_figura.NOMBRE)
        self.geometry(f"280x{225 + 30 * n}+{padre.winfo_x() + 30}+{padre.winfo_y() + 30}")
        self.resizable(False, False)
        self.configure(bg=COLOR_FONDO)
        lienzo = tk.Canvas(self, width=280, height=100, bg=COLOR_FONDO, highlightthickness=0)
        lienzo.place(x=0, y=5)
        clase_figura.dibujar(lienzo)
        self.campos = []
        for i, dato in enumerate(clase_figura.DATOS):
            y = 120 + 30 * i
            tk.Label(self, text=f"{dato}:", bg=COLOR_FONDO, anchor="w").place(x=20, y=y,
width=100, height=23)
            campo = tk.Entry(self)
            campo.place(x=120, y=y, width=135, height=23)
```

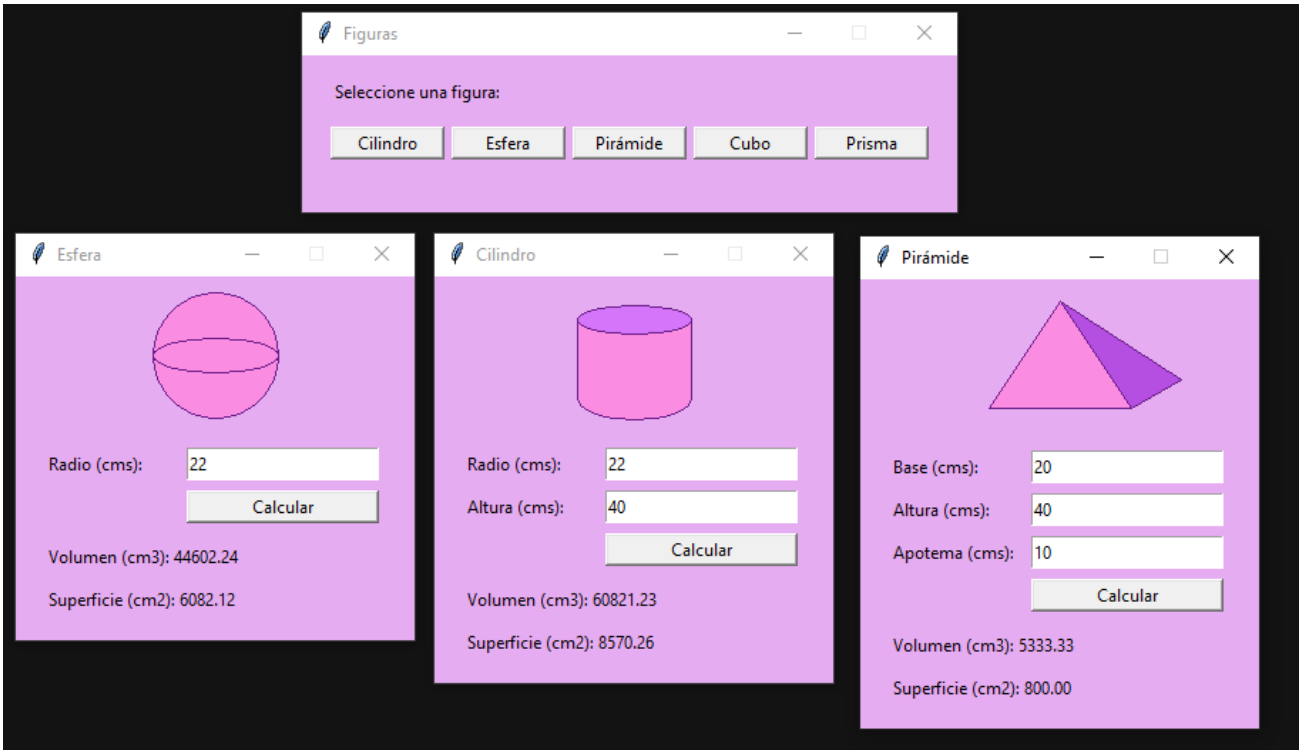
```
            self.campos.append(campo)
        y = 120 + 30 * n
        tk.Button(self, text="Calcular", command=self.calcular).place(x=120, y=y, width=135,
height=23)
        self.volumen = tk.Label(self, text="Volumen (cm3):", bg=COLOR_FONDO, anchor="w")
        self.volumen.place(x=20, y=y + 35, width=240, height=23)
        self.superficie = tk.Label(self, text="Superficie (cm2):", bg=COLOR_FONDO, anchor="w")
        self.superficie.place(x=20, y=y + 65, width=240, height=23)

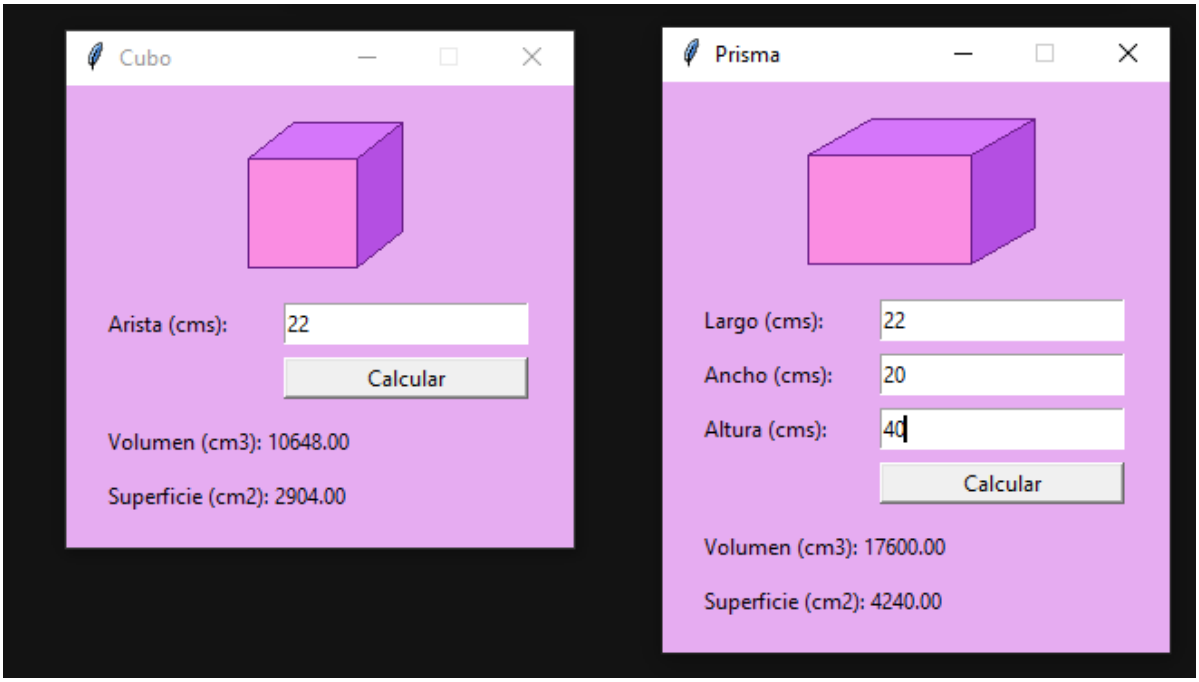
    def calcular(self):
        try:
            valores = [float(c.get().strip().replace(",", ".")) for c in self.campos]
            if not all(math.isfinite(v) and v > 0 for v in valores):
                raise ValueError
        except ValueError:
            messagebox.showerror("Error", "Campo vacío o error en formato de número.\n""Ingrese
números mayores que 0.", parent=self)
            return
        figura = self.clase_figura(*valores)
        self.volumen.config(text=f"Volumen (cm3): {figura.volumen:.2f}")
        self.superficie.config(text=f"Superficie (cm2): {figura.superficie:.2f}")

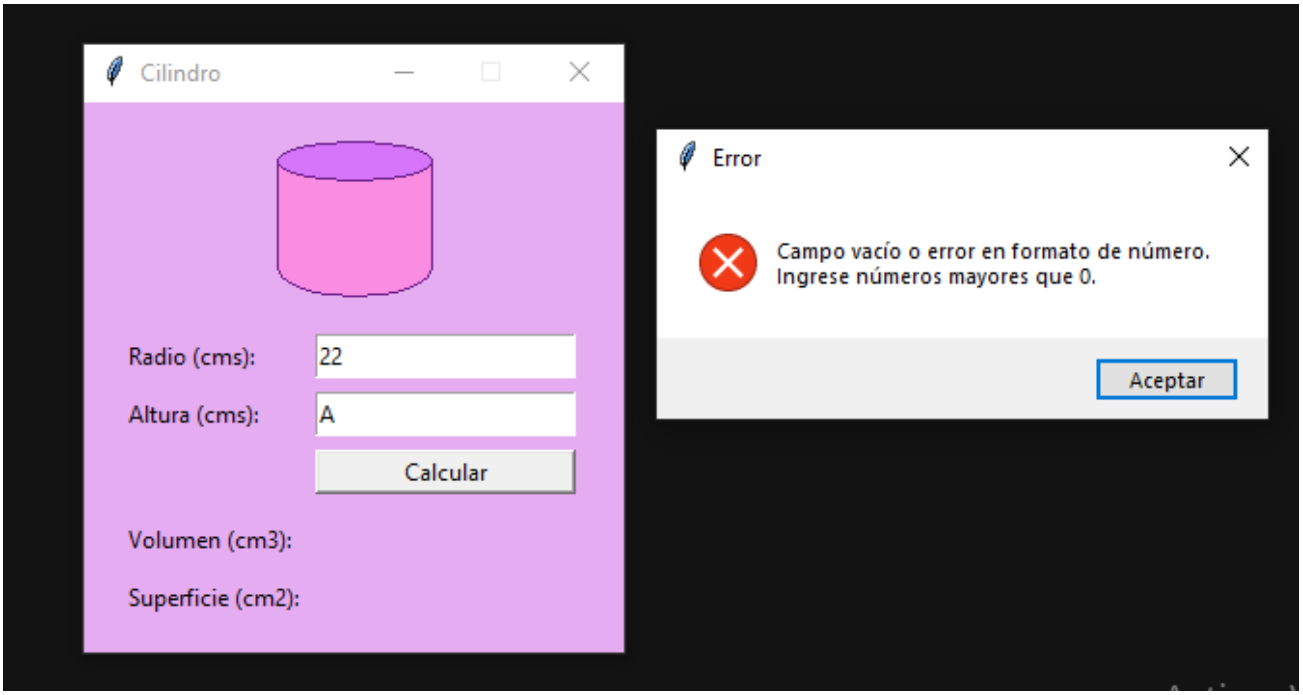
class VentanaPrincipal(tk.Tk):
    FIGURAS = [Cilindro, Esfera, Piramide, Cubo, Prisma]
    def __init__(self):
        super().__init__()
        self.title("Figuras")
        self.geometry("460x110")
        self.resizable(False, False)
        self.configure(bg=COLOR_FONDO)
        tk.Label(self, text="Seleccione una figura:", bg=COLOR_FONDO).place(x=20, y=15)
        for i, clase in enumerate(self.FIGURAS):
            tk.Button(self, text=clase.NOMBRE, command=lambda c=clase: self.abrir(c)).place(x=20 + i
* 85, y=50, width=80, height=23)
    def abrir(self, clase_figura):
        VentanaFigura(self, clase_figura)

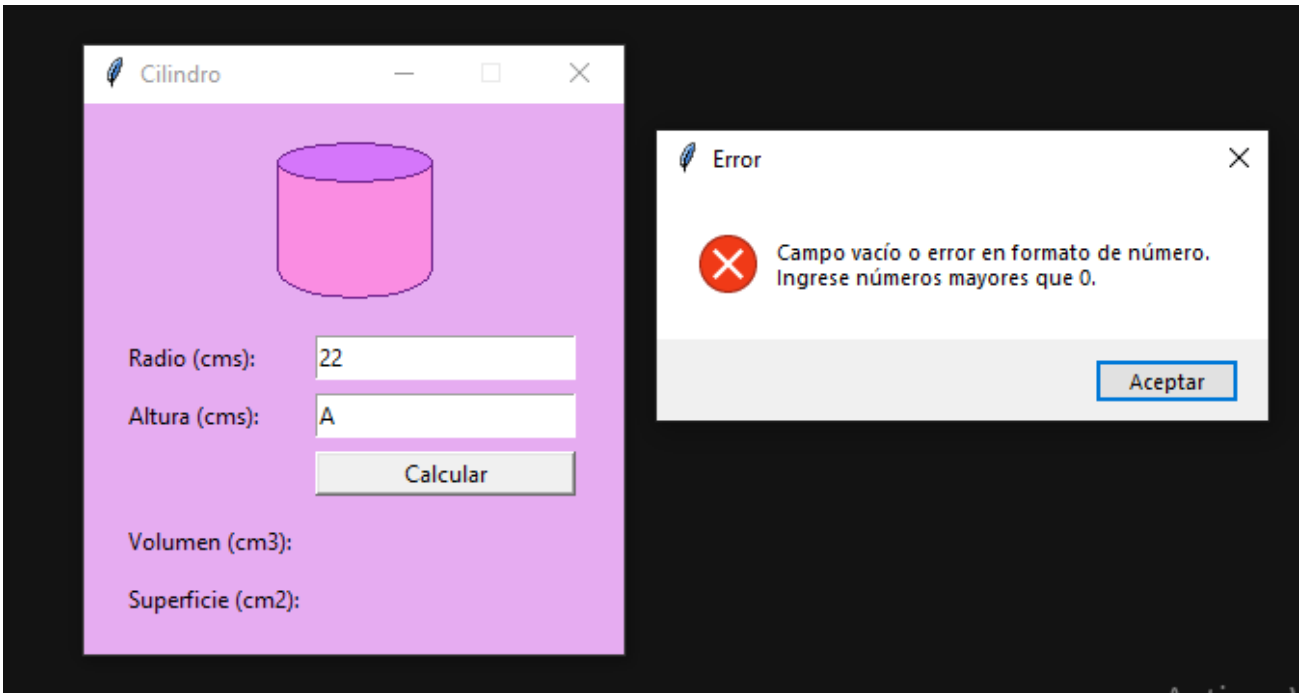
class Principal:
    @staticmethod
    def main():
        VentanaPrincipal().mainloop()

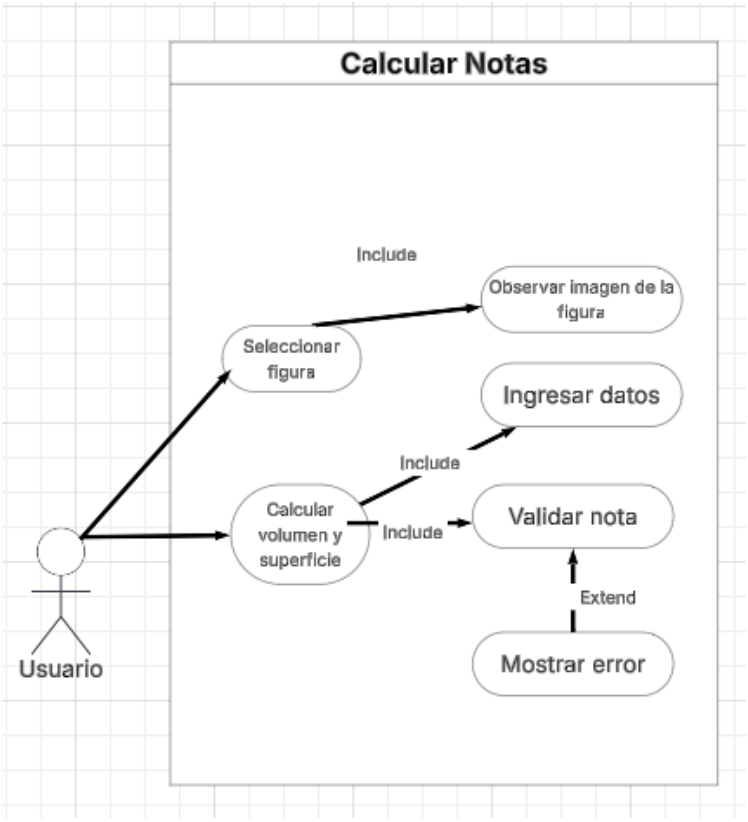
if __name__ == "__main__":
    Principal.main()
```

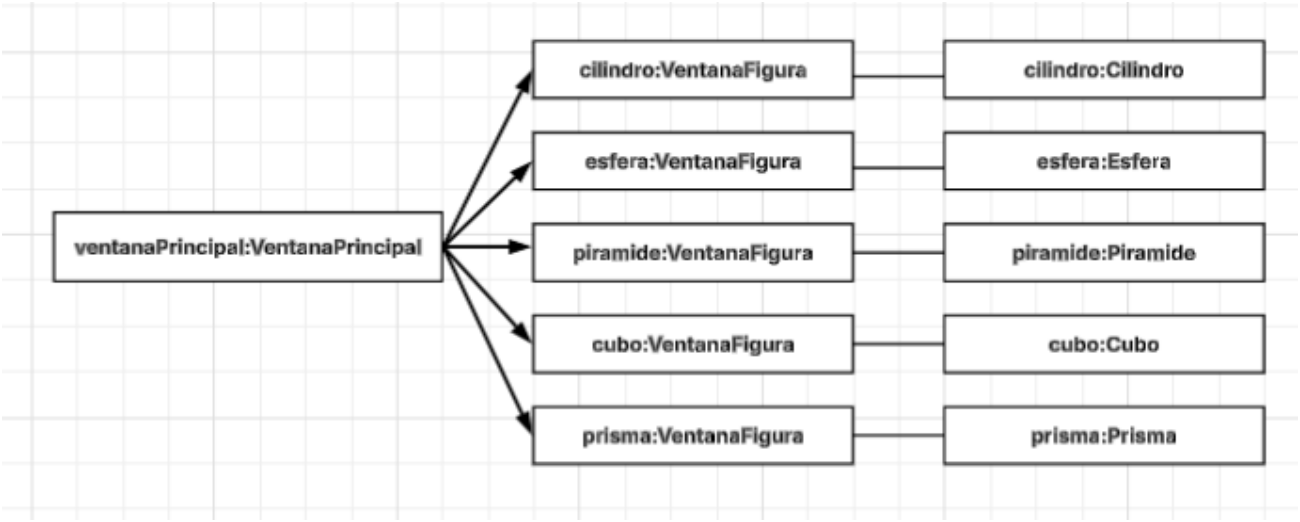

Figuras
Seleccione una figura:
Cilindro
Esfera
Pirámide
Cubo
Prisma
Esfera
Radio (cms): 22
Calcular
Volumen (cm3): 44602.24
Superficie (cm2): 6082.12
Cilindro
Radio (cms): 22
Altura (cms): 40
Calcular
Volumen (cm3): 60821.23
Superficie (cm2): 8570.26
Pirámide
Base (cms): 20
Altura (cms): 40
Apotema (cms): 10
Calcular
Volumen (cm3): 5333.33
Superficie (cm2): 800.00


Cubo
Arista (cms): 22
Calcular
Volumen (cm3): 10648.00
Superficie (cm2): 2904.00
Prisma
Largo (cms): 22
Ancho (cms): 20
Altura (cms): 40
Calcular
Volumen (cm3): 17600.00
Superficie (cm2): 4240.00


Cilindro
Radio (cms):
22
Altura (cms):
A
Calcular
Volumen (cm3):
Superficie (cm2):
Error
Campo vacío o error en formato de número.
Ingrese números mayores que 0.
Aceptar


Cilindro
Radio (cms):
22
Altura (cms):
A
Calcular
Volumen (cm3):
Superficie (cm2):
Error
Campo vacío o error en formato de número.
Ingrese números mayores que 0.
Aceptar


Calcular Notas
Include
Observar imagen de la figura
Seleccionar figura
Ingresar datos
Include
Calcular volumen y superficie
Include
Validar nota
Extend
Mostrar error
Usuario


ventanaPrincipal:VentanaPrincipal
cilindro:VentanaFigura
cilindro:Cilindro
esfera:VentanaFigura
esfera:Esfera
piramide:VentanaFigura
piramide:Piramide
cubo:VentanaFigura
cubo:Cubo
prisma:VentanaFigura
prisma:Prisma


VentanaPrincipal
+FIGURAS:list[type]
+ __init__()
+abrir(clase_figura)
VentanaFigura
+clase_figura:type
+campos:list[Entry]
+volumen:Label
+superficie:Label
+ __init__()
+calcular()
Principal
+main()
FiguraGeometrica
+NOMBRE:str
+DATOS:list[str]
+volumen:float
+superficie:float
+calcular()
+calcular_volumen()
+calcular_superficie()
+dibujar(lienzo)
Cilindro
+radio:float
+altura:float
+ __init__()
+calcular_volumen()
+calcular_superficie()
+dibujar(lienzo)
Esfera
+radio:float
+ __init__()
+calcular_volumen()
+calcular_superficie()
+dibujar(lienzo)
Piramide
+base:float
+altura:float
+apotema:float
+ __init__()
+calcular_volumen()
+calcular_superficie()
+dibujar(lienzo)
Cubo
+arista:float
+ __init__()
+calcular_volumen()
+calcular_superficie()
+dibujar(lienzo)
Prisma
+largo:float
+ancho:float
+altura:float
+ __init__()
+calcular_volumen()
+calcular_superficie()
+dibujar(lienzo)